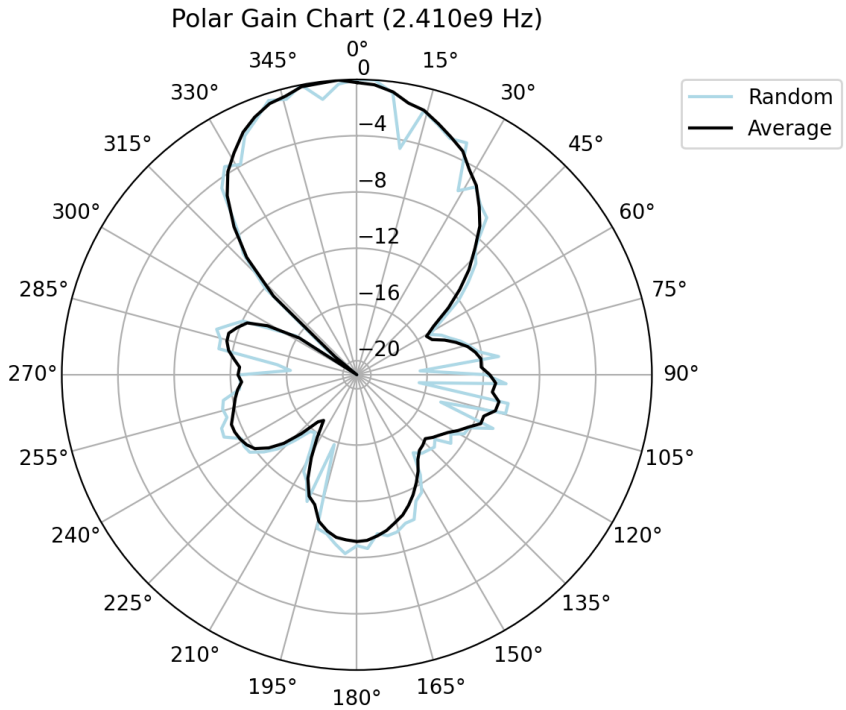
Open Source RF Measurement System

A senior capstone project by Stephen Harris and Jared Bell.

Sponsored by Dr. Sai Radavaram at Utah Tech University.



The Open Source RF Measurement System is an open source project to measure, record, and plot the gain of an antenna over a radial access. Provided in the github repository (located here: https://github.com/stephen010x/capstone-rfsweep/tree/main) are the 3D printed cad models, the PCB design, and all of the software used for this project.

Table of Contents

- How to Download
- Connecting to the RFSWEEP server
  - From the PI's Access Point (intended)
  - From `localhost`
  - From a Local Connection
  - From a Public Connection
  - SSH into the Raspberry PI Controller
- How to Use Scripts
  - Interactive Receiver Script
  - Interactive Transmitter Script
  - Fastrun Script
  - How to use `process.py`
- How to use `rfsweep`
  - `rfsweep` Commands
  - `rfsweep` Parameters
  - `rfsweep` General Parameters
  - `rfsweep ping` Parameters
  - `rfsweep server` Parameters
  - `rfsweep reset` Parameters
  - `rfsweep restart` Parameters
  - `rfsweep getlogs` Parameters
  - `rfsweep transmit` Parameters
  - `rfsweep measure` and `rfsweep receive` Parameters
  - `rfsweep rotate` Parameters
  - Supported baseband filters
- `rfsweep` Data Formats
- Hardware Notes
- CAD Models
- Building `rfsweep`
  - Compile for Debian/Ubuntu
  - Compile on Windows CMD
  - Compile on Windows Desktop
  - Cross-Compile for Windows from Debian/Ubuntu
- Installing Script Dependancies (optional)
  - Debian/Ubuntu Script Dependancies
  - Windows CMD Script Dependancies
  - Windows Desktop Script Dependancies
- Resources and Documentation

How to Download

You can download the binaries and scripts neccessary for using `rfsweep` for both Windows and Debian/Ubuntu in the releases page here.

Connecting to the RFSWEEP server

Communication is managed wirelessly or through ethernet. Both the client and server must have some way to connect to each-other. The connection can either be wireless, or through ethernet. In order for communication to happen, the client has to know the IP address of the server.

On the prototype, the default Wi-Fi and Ethernet IP address is often either `10.42.0.1`, `10.42.1.1`, or `10.42.x.1`, and the default port is `7070`

From the PI's Access Point (intended)

- The Raspberry PI is configured to broadcast a wireless access point that you can connect to called raspi. On the prototype, connecting to the `raspi` network requires no password.

- Note that you may lose connection to the internet when connecting to the Raspberry PI. Generally it is an accepted risk to loose internet access while taking measurements. However, if you want internet access while connected to raspi, you can enable that by connecting the Raspberry PI controller to the internet by plugging in a USB Wi-Fi dongle to the USB hub connected to the Raspberry PI, and then accessing the PI through either `ssh` (See [SSH into the Raspberry PI Controller]) or through other means. And then connecting the PI to Wi-Fi from a network utility such as `nmtui`.

From `localhost`

- If the server is running from the same computer as the client, you can connect them via the localhost IP address `127.0.0.1`.

From a Local Connection

- If both the client and the server are connected to the same local network, you can connect to the PI using it's local IP address.

From a Public Connection

- Connecting to the Raspberry PI through a public connection can be more tricky. First, you have to know the public IP address of the router that the PI is connected to. Next, you have to make sure the port (default is port `7070`) the server is using is exposed on the router. (The default on most routers is to firewall all ports).

SSH into the Raspberry PI Controller

- The Raspberry PI is a headless controller. As a result, the easiest and most convenient way to access it is through SSH. Through SSH, you are able to access the PI directly through the command line.

- By default, the PI's wireless access point IP address is `10.42.0.1`. To SSH into the PI make sure you are connected to the raspi network

broadcasted by the PI. (If you have only just turned on the PI, wait about 60 seconds for the device to boot up and configure the network). Then you can run `ssh user@10.42.0.1:/home/user` with password `pass`.

- For the prototype, both the Raspberry PI's `root` and `user` password has been set to `pass`.

- In the case of the PI's Access Point or Wi-Fi connection malfunctioning, you are able to take out it's SD memory card, and edit the `startup.sh` script in `/home/user/`. The startup script in the home user directory is ran whenever the PI boots up, and requires some technical competency in GNU/Linux command line and Bash scripting. This script is responsible for setting up Wi-Fi and autostarting the `rfsweep` server.

- If you wish to modify the Wi-Fi startup script, deleting the `key/` directory in the `user` home directory will trigger the Wi-Fi script to reconfigure the Access Point.

How to Use Scripts

See [Installing Script Dependancies (optional)] for how to install the dependancies required to run these scripts.

Provided with the rfsweep binary are four scripts. `run.bat`, `fastrun.bat`, `transmit.bat`, and `process.py` for Windows. Or `run.sh`, `fastrun.sh`, and `process.py` for Linux.

For the interactive transmitter and receiver scripts: If the IP address is left blank, these scripts will test for the known default access point address, followed by the known default ethernet address, followed by the localhost address.

Also provided is the `process.py` script, which is the default script used to process and graph the data returned from `rfsweep`.

Interactive Receiver Script

`run.bat` and `run.sh` act as an interactive script that will prompt you for the receiver parameters. See [`rfsweep measure` and `rfsweep receive` Parameters] for more information on the measurement parameters.

Once the script ends, it will run process.py, which will process and graph the data received.

Interactive Transmitter Script

`transmit.bat` is an interactive script that will prompt you for the transmitter parameters. See [`rfsweep transmit` Parameters] for more information on the transmitter parameters.

▶ For instance:

Fastrun Script

The `fastrun.bat` and `fastrun.sh` scripts are non-interactive scripts that are meant to be edited. Inside they have variables that you can modify to allow for a quick, easy, and repeatable way to run tests without having to use the interactive scripts.

These scripts manage both the transmitter and receiver. When finished it runs `process.py`, which will process and graph the data received.

▶ Variables Intended to be Modified:

How to use `process.py`

`process.py` is a python script that is packaged alongside the binaries by default. This script is an optional companion script to the `rfsweep` command, and provides for an easy way to process and visually plot the

data received by running `rfsweep measure` or `rfsweep receive`. See [How to use rfsweep]() for more information on how to use `rfsweep`.

The `process.py` help manual is available by running:

```
python3 process.py -h
```

To use `process.py`, append one or more file names for it to process. Files can be in binary format, as well as text format. `process.py` detects these formats via file extensions such as `.bin` and `.txt`. See [rfsweep Data Formats]() for more information on the accepted formats.

Example usage:

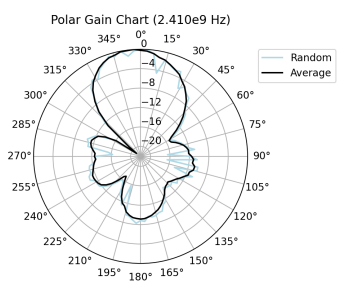
```
python3 process.py data1.bin data2.txt
```

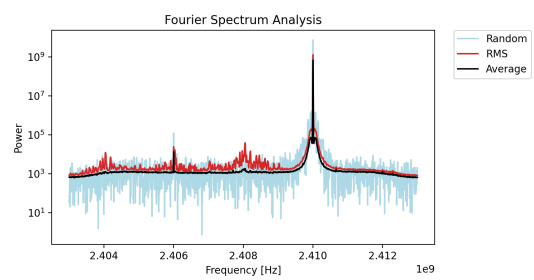
`process.py` is able to automatically determine the bandpass filter frequency by looking for the peak signal across all of the data points. However, this may be unreliable with messier signals. If you wish to manually specify a filter frequency, use the `--freq <value>` flag.

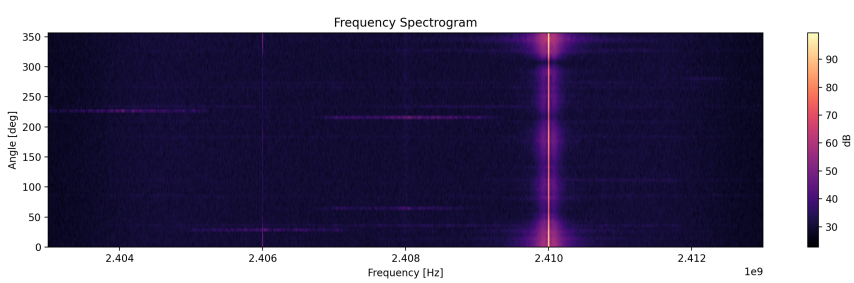
```
python3 process.py data.bin --freq 2.41e9
```

WARNING: Be wary of using `process.py` for large data files. `process.py` uses a substantial amount of memory to process large data files. Data output from `rfsweep` can range from tens of Megabates, to hundreds of Megabytes. And this gets multiplied by Python's memory usage. Processing data files larger than 200 MB can result in `process.py` memory usage of 5 GB or more!

Here are examples of data that was processed using `process.py`







How to use `rfsweep`

`rfsweep` is a command line tool with various options that allows the user to wirelessly send commands to the Raspberry PI controller that manages the motor and antennas. The communication is server/client based over TCP. Both the client and the server must be connected via Wi-Fi or Ethernet.

Within the command line, you can get a help manual by running:

```
rfsweep -h
```

See the following [`rfsweep` Commands]() or [`rfsweep` Parameters]() for more detailed usage.

`rfsweep` Commands

`rfsweep` has several commands that can be sent to the server. You can view these commands by running:

```
rfsweep -h
```

Each command also has their own help manual:

```
rfsweep <command> -h
```

The commands are as listed below:

```
COMMANDS:
        ping        Ping server.
        server      Start server.
        reset       Reset remote server.
        restart     Restart remote server system.
        getlogs     Return remote server error logs.
        transmit    Enable/disable remote server transmitter.
        measure     Run remote server measurements.
        receive     Run measurements without rotation.
        rotate      Move motor to indicated angle.

USAGE:
        rfsweep -h
        rfsweep <command> -h
        rfsweep <command> [options]
```

`rfsweep` Parameters

Parameters are flags that typically follow after a command, that allows us to configure the behavior when the server receives the command. They typically follow the format `--<param>=<value>`

For instance,

```
rfsweep rotate --angle=360
```

All parameters have a default value. All You can query for the default parameters by running:

```
rfsweep --defaults
```

Note that leaving parameters blank will result in them using their default value. Like so:

```
rfsweep ping --ip= --port=
```

Most parameters are specific to certain commands, as will be explained in the following sections.

`rfsweep` General Parameters

These are parameters that apply to every command except the server command.

```
OPTIONS
        -h
            Print help message.

        -v
            Verbose. Echos parameters.

        --ip=<ip>
            Server ip address.

        --port=<port>
            Server port number.

DEFAULTS:
        --ip=10.42.0.1
        --port=7070

USAGE:
        rfsweep <command> -h
        rfsweep <command> [options]
```

NOTES:

The `-v` flag prints out the parameters being used. This tells us exactly what parameters is being sent to the server, and is useful for detecting whether or not we have passed `rfsweep` bad parameters.

The `--ip` and `--port` flags set the IP address and PORT that points to the server. If these are wrong, then the client will fail to send any messages to the server.

`rfsweep ping` Parameters

The `ping` command sends a probing signal to the server to determine if the server is reachable. If this command fails, it means that either A) The server is inactive, or B) The server is unreachable, which typically occurs when the server host computer or controller is off, or if your IP or PORT are wrong.

If `ping` fails, check that the Raspberry PI is on, that you are connected to the raspi Wi-Fi SSID or through an Ethernet connection, and finally make sure that your IP address and PORT are correct.

See [`rfsweep` General Parameters]() for rfsweep ping parameters.

`rfsweep server` Parameters

The `server` command is a special mode that behaves unlike any of the other commands. The other commands are clientside commands being sent to the server. Running `rfsweep server` will start a server instance that is able to receive any commands sent by `rfsweep` client commands.

The prototype is configured to autostart this on the Raspberry PI.

```
DESCRIPTION
        Start server.

OPTIONS
        -h
            Print help message.

        -v
            Verbose. Echos parameters.

        --log=<filepath>
            Path to log file.

        --port=<port>
            Port to listen to.

        --tserial=<serial>
            Transmission HackRF serial string.

        --rserial=<port>
            Receiver HackRF serial string.

DEFAULTS:
        --port=7070
        --log=NULL
        --rserial="0000000000000000d07864dc314d3287"
        --tserial="0000000000000000930c64dc285b0ec3"

USAGE:
        rfsweep server -h
        rfsweep server [options]
```

NOTES:

The `--log` flag sets the path to the log file that the server will return when the client calls `rfsweep getlogs`.

The `--tserial` and `--rserial` flags are the serial number strings for identifying the transmitter HackRF, and the receiver HackRF. These serial numbers are unique to each HackRF, and are how the server indentifies which HackRF is the receiver, and which is the transmitter. As a consiquence, using different HackRFs will require you to obtain the serial number, and specifying it in the flags. You can obtain the serial number by running `hackrf_info` from the HackRF Tools library, which can be

found in the `hackrf-tools\` directory of the Windows build. Otherwise `hackrf_info` can be obtained through Debian/Ubuntu's `apt` or by following the instructions on the HackRF Docs: https://hackrf.readthedocs.io/en/latest/installing_hackrf_software.html.

`rfsweep reset` Parameters

The `rfsweep reset` command will tell the server to restart it's running process. If the `startup.sh` is not running `rfsweep` server on loop, then it will not restart, and will only kill the process.

See `rfsweep` General Parameters for rfsweep ping parameters.

`rfsweep restart` Parameters

The `rfsweep restart` command will tell the server to reboot the computer that it is running on, which typically should be the Rasberry PI controller.

See `rfsweep` General Parameters for rfsweep ping parameters.

`rfsweep getlogs` Parameters

The `rfsweep getlogs` command will request the error logs from the server, and print it to stdout.

See `rfsweep` General Parameters for rfsweep ping parameters.

`rfsweep transmit` Parameters

The `rfsweep transmit enable` and `rfsweep transmit disable` command allows you to enable or disable the transmitter. When enabled, the transmitter will generate a sinusoidal wave on the specified frequency.

```
DESCRIPTION
        Enables or disables the transmitting HackRF controlled
        by the server. Emits a constant frequency across the
        specified bandwidth.

OPTIONS
        -h
            Print help message.

        -v
            Verbose. Echos parameters.

        --ip=<ip>
            Server ip address.

        --port=<port>
            Server port.

HACKRF OPTIONS
        --freq=<freq_hz>
```

```
            Set center frequency (hz).

        --vga-gain=<gain_db>
            Variable gain amplifier. Software ampllification of
            the signal.
            Steps of 1 dB. Ranges from 0 to 47 dB

        --tx-ampl=<amplitude>
            Amplitude of transmission relative to gain (0-127)

        --amplify
            Enable amplifier.

        --clock
            Enable transmitter clock out enable.

DEFAULTS
        --ip=10.42.0.1
        --port=7070
        --freq=2.4e9
        --vga-gain=20
        --tx-amp=127

USAGE:
        rfsweep transmit enable  [options]
        rfsweep transmit disable [options]
```

A typical use of this command would look like this:

```
rfsweep transmit enable -v --binary --file="data.bin" --freq=2.41e9 --vga-gain=20
```

Don't forget to disable when measurements are complete:

```
rfsweep transmit disable
```

NOTES:

The HackRF has a "clock in" and a "clock out" coaxial port on it's body. Hooking the "clock out" of one HackRF to the "clock in" of another HackRF, and enabling the clock with the `--clock` flag will syncronize the receiver and transmitter.

The HackRF will automatically detect if it receives a clock signal on the "clock in" coaxial port. Therefore, you only need to enable the clock on for one of the HackRF boards. (in this case the transmitter is recommended).

For more information on the HackRF One External Clock Interface, see these:
https://hackrf.readthedocs.io/en/latest/external_clock_interface.html
https://hackrf.readthedocs.io/en/latest/hardware_triggering.html

`rfsweep measure` and `rfsweep receive` Parameters

The `rfsweep measure` and `rfsweep receive` are the core commands of `rfsweep` that allows us to record measurements to later be processed.

These commands are nearly identical. They both request the server to perform measurements, and return the data back to the client. The only

difference between the two is that `rfsweep measure` will take measurements while rotating, whereas `rfsweep receive` will take measurements while remaining stationary.

```
DESCRIPTION
        Request to server to run measurements, returning them
        to stdout or to a specified file.

        Data is formatted like so:
        <timestamp_micro:f64> <angle:f64> <freq_hz:f64>
        <band_hz:f64> <samplerate_hz:f64> <bytecount:i64> [
        <real:i8> <imaginary:i8> ...]

OPTIONS
        -h
            Print help message.

        -v
            Verbose. Echos parameters.

        --ip=<ip>
            Server ip address.

        --port=<port>
            Server port.

        --file=<filepath>
            Output server response to file rather than stdout.

HACKRF OPTIONS
        --freq=<freq_hz>
            Set center frequency (Hz).

        --band=<band_hz>
            Set baseband filter bandwidth (Hz). Will round the
            frequency to the nearest supported frequency.

        --srate=<srate_hz>
            Set sample rate (Hz).

        --lna-gain=<gain_db>
            Low noise amplifier. Hardware amplification of small
            signals. Steps of 8 dB. Ranges from 0 to 40 dB

        --vga-gain=<gain_db>
            Variable gain amplifier. Software ampllification of
            the signal to adjust 8-bit saturation
            Steps of 2 dB. Ranges from 0 to 62 dB

        --steps=<steps>
            Number of angles to take samples (divided across
            360 degrees.

        --samps=<samples>
            Number of chunk samples to take per angle. (chunks
            are usually around 65,536 interleaved bytes of data
            each)

        --stepmode=<1/2/4/8/16>
            Set motor microstep resolution.
```

```
        --amplify
            Enable amplifier.

        --binary
            Output results in binary instead of ascii.

        --clock
            Enable receiver clock out enable.

DEFAULTS
        --ip=10.42.0.1
        --port=7070
        --samps=1
        --steps=360
        --freq=2.4e9
        --srate=10e6
        --band=<srate*0.75>
        --lna-gain=16
        --vga-gain=20
        --stepmode=1

USAGE:
        rfsweep measure [options]
        rfsweep receive [options]
```

A typical use of this command would look like this:

```
rfsweep measure -v --binary --file="data.bin" --freq=2.41e9 --lna-gain=16 --vga-
          gain=20
```

NOTES:

You can set the baseband filter frequency with the `--band` flag. The baseband filter frequency will be rounded down to the nearest supported baseband frequencies. To see supported baseband filter frequencies, see [Supported baseband filters]() for more information.

If `--band=` is left blank, then the bandwidth will be set to `0.75 * srate` by default. To disable the baseband filter, simply set it to either zero `--band=0` or any number larger than `28 MHz` e.g. `--band=30e6`.

WARNING: Setting the baseband filter `--band` higher than the sample rate `--srate` will result in aliasing!

Note that you will often have to tune the `--lna-gain` and the `--vga-gain` in order to maximize the accuracy of your measurements, while also minimizing saturation. The LNA gain is the Low-Noise Amplifier, which amplifiees low-power signals without degrading its signal-to-noise ratio. On the other hand the VGA gain is the Variable-Gain amplifier, which will amplify both your signal and your noise. However, it is useful to prevent your signal from saturating, as the signal must fit within a signed 8 bits (-128 to 127).

When running `process.py` on a sample, it will output a count of how many likely signal saturations there are in your data, and print it to stdout. Using this can help you tune your VGA and LNA gain.

More information on the LNA gain and the VGA gain here:
https://en.wikipedia.org/wiki/Low-noise_amplifier
https://en.wikipedia.org/wiki/Variable-gain_amplifier \

WARNING: A caution with the `--samps` parameter. Each sample is approximately 64 KiB each. The size of the data in bytes can be calculated as such: `65584 * samps * steps`, which can grow fast. If you are taking 10 samples at 360 steps per revolution, the output data will be around 225 MiB, which is a rather substantial file size. And that is only with a binary file. If you are outputting ASCII by omitting the `--binary` flag, the file size can be multiplied by 3-5 times in size! Furthermore, if you are passing data to `process.py`, `process.py` consumes a lot of memory. Sometimes upwards of 5 GB or more depending on the size of your data file. So be cautious of taking too many samples. In many cases 1 to 3 samples is enough. By default it is set to 1.

Finally, make sure to either specify an output file with `--file`, or redirect the output to a file like so: `rfsweep measure >> data.txt`. Otherwise huge amounts of data will be dumped to your stdout.

`rfsweep rotate` Parameters

`rfsweep rotate` allows us to send rotation commands to the controller. It can be useful in conjunction with `rfsweep receive` or for other tests.

```
DESCRIPTION
        Move motor to indicated angle.

OPTIONS
        -h
            Print help message.

        -v
            Verbose. Echos parameters.

        --angle=<degrees>
            Set motor angle in terms of degrees

        --steps=<degrees>
            Set motor angle in terms of steps

        --stepmode=<1/2/4/8/16>
            Set motor microstep resolution.

DEFAULTS (--defaults)
        --ip="10.42.0.1"
        --port=7070
        --steps=360
        --angle=0
        --stepmode=1

USAGE:
        rfsweep rotate [options]
```

NOTE:

Either specify `--angle` or specify `--steps`. Not both. They both indicate where the motor should rotate to. `--angle` is often only approximate, but will get the motor as close as possible to that angle, whereas `--steps` is exactly how many steps from origin the motor should rotate to.

A direct conversion looks like this: $\theta_{deg}= \frac{33}{95\cdot S_{tepmode}}\cdot S_{teps}$

WARNING: Take care to not rotate too far or the coaxial cable to the antenna may get twisted.

Supported baseband filters

Here is a list of supported baseband filters:

```
    1.75 MHz
    2.50 MHz
    3.50 MHz
    5.00 MHz
    5.50 MHz
    6.00 MHz
    7.00 MHz
    8.00 MHz
    9.00 MHz
   10.00 MHz
   12.00 MHz
   14.00 MHz
   15.00 MHz
   20.00 MHz
   24.00 MHz
   28.00 MHz
```

`rfsweep` Data Formats

`rfsweep` has two primary data formats. Binary, and ASCII. They both follow a similar structure. However the binary format is much more compact and generally faster to process, whereas the ASCII is more human readable and potentially easier to process.

By default `rfsweep` outputs ASCII to stdout. However you can have it output binary instead with the `--binary` flag.

ASCII FORMAT:

When outputting to ASCII, the very first line will be a format guide, and should be ignored when processing.

The ASCII output format are space separated values, or SSV. And then each new line is a different sample taken. Each sample is began with 6 metadata values, followed by data that is typically 65,536 items long,

Each data value is interleaved real/imaginary bytes. The first byte is real, the second is imaginary, the third is real, etc.

For instance,

```
timestamp(us) angle(degrees) freq(Hz) bandwidth(Hz) samplerate(Hz) bincount [real imag
...]
9116512597 36.1263 2.4e+09 7e+06 1e+07 65536 37 74 190 41 192 191 35 163 97 249 31 74
189 37 195 187 40 162 95 254 26 75 188 32 197 186 43 163 95 255 25 78 188...
```

The first six values of each sample is the timestamp in microseconds, the angle in degrees, the frequency in hertz, the bandwidth filter in hertz, and the number of data items, followed by the data.

For the ASCII format, the bincount is typically uneccessary, as each sample is separated by a newline. (though it can be used to detect if there was a data mismatch, or data cut off too early)

BINARY FORMAT:

The binary format follows just like the ASCII format, only more compact. And interpreting it can be tricky without the right tools.

Here is an example of the first 80 bytes:

```
00 00 00 8B CF 7C B8 41 00 00 00 E0 1A CA 0B 40 00 00 00 40
E5 F0 E1 41 00 00 00 00 F0 B3 5A 41 00 00 00 00 D0 12 63 41
00 00 01 00 00 00 00 00 5F F1 27 4A C2 2A BE C2 20 A1 61 F4
25 4A BE 29 C0 BF 23 A3 61 F9 1F 4A BD 25 C3 BB 28 A2 5F FE
```

These can be broken down like so:

```
[ 00 00 00 8B CF 7C B8 41 ]     timestamp   - 64 bit float
[ 00 00 00 E0 1A CA 0B 40 ]     angle       - 64 bit float
[ 00 00 00 40 E5 F0 E1 41 ]     frequency   - 64 bit float
[ 00 00 00 00 F0 B3 5A 41 ]     bandwidth   - 64 bit float
[ 00 00 00 00 D0 12 63 41 ]     samplerate  - 64 bit float
```

```
[ 00 00 01 00 00 00 00 00 ]     bincount    - 64 bit signed integer
[ 5F ]      real  - 8 bit signed integer
[ F1 ]      imag  - 8 bit signed integer
[ 27 ]      real  - 8 bit signed integer
[ 4A ]      imag  - 8 bit signed integer
  ...
```

It is important to note that all of this data is Little Endian. Which basically means the bytes are reversed. Visit here for more on Endianness: https://en.wikipedia.org/wiki/Endianness

Unlike the ASCII format, there is no newline to tell you where the sample ends. As such, you will need to use the bincount to tell when the data ends. Otherwise, it will completely mess up your interpreter for all bytes after.

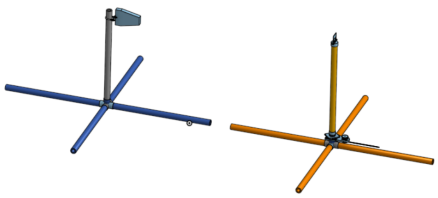
Hardware Notes

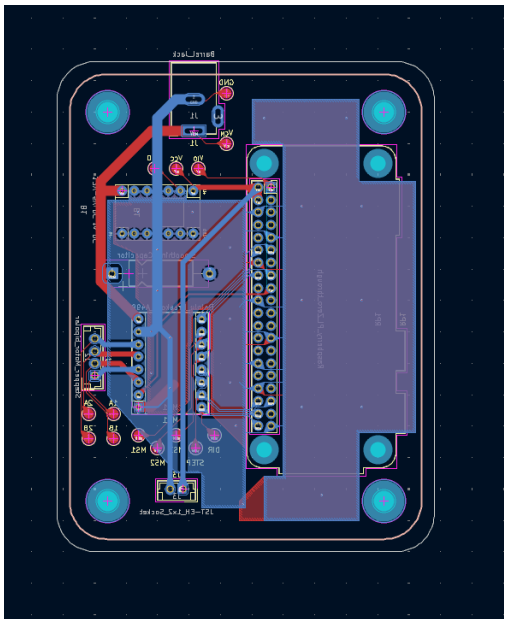
1. DC blocks need to be installed between each HackRF and Antenna. If you get a "USB Error," this is likely the cause.
   1. If the amplifier is connected, it acts as a dc block, even if it is turned off.
   2. If the amplifier is turned off but connected in place of dc block, it will still pass rf signals, though less effectively than the dc blocks.
2. When feeding coaxial cables through the receiver antenna stand, it is set up to be able to easily disconnect the physical parts for feeding the cable.
3. If the rotation assembly seems noisier than usual, the set screw for the pulley on top of the motor is likely loose.
   1. Removable loctite was used to try and retain the set screw. The loctite may need to be reapplied.
   2. If the graph of the signal looks significantly different than expected, also check the set screw.
4. While most of the antennas used have an sma connector, the monopoles are rp-sma. Rp-sma is threaded opposite the sma connectors, so it will appear to fit, but have no internal connection.
5. 3d printed antenna holders are designed such that the slit in the bottom is for feeding the coaxial cable through for strain relief

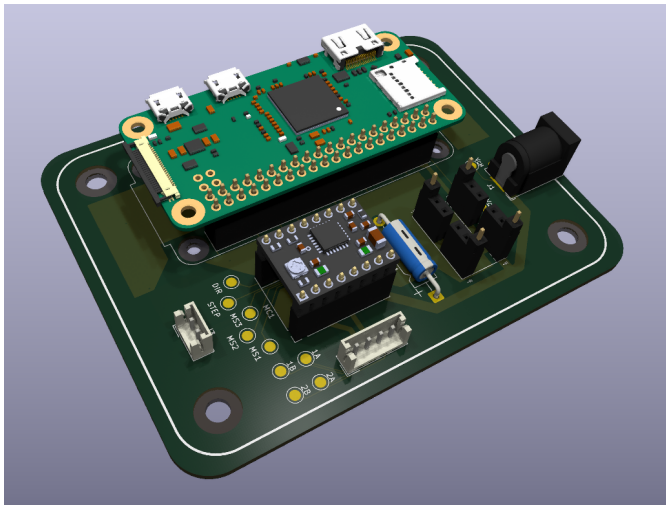
1. Transmitter and receiver HackRF's are noted as such on their cases. The software program built for this specific project requires the same HackRF's to be connected to transmitter and receiver every time (unless the `--rserial` or `--tserial` flags are used). If using any external software, the receiver and transmitter requirements do not apply.
2. GNU Radio, part of radioconda, is one of the only external tools that supports HackRF transmission, should you elect to transmit with

software outside of our custom built application
    1. Two blocks were used by us to transmit:
        1. Constant Source block as the signal source
        2. Soapy HackRF Sink to take the input from the constant source
    2. Apart from GNU Radio and our custom software, using the command line yourself is the other way to directly transmit with the HackRF.
3. SDR Console is an external program that easily connects to the HackRF to observe current ambient rf signals. It does not have support for HackRF transmission.
4. All 3d prints were made with grey petg using 15% infill
5. Direct carbon fiber fabric was the only non-metal found to be able to block RF radiation.

- CAD Models







The CAD models for the PCB board and the 3D printed parts can be found here: https://github.com/stephen010x/capstone-rfsweep/tree/main/cad/

Building `rfsweep`

Compile for Debian/Ubuntu

Tested on Debian (Trixie) and Ubuntu (Noble Numbat)

```
# Update apt
sudo apt update

# Install dependancies
sudo apt-get install git gcc make libusb-1.0-0 libusb-1.0-0-dev

# Clone the project repository
git clone https://github.com/stephen010x/capstone-rfsweep.git --recurse-submodules
cd capstone-rfsweep

# If cloned without submodules, you can update the submodules like so:
git submodule init
git submodule update

# Build project
make

# Once complete, the compiled output can be found in 'bin/'
```

Compile on Windows CMD

```
# Open the windows command line

# Install git if not already installed
winget install Git.Git

# Clone the project repository
git clone https://github.com/stephen010x/capstone-rfsweep.git
cd capstone-rfsweep

# Install cygwin
call install_cygwin.bat

# Build project
make

# Once complete, the compiled output can be found in 'bin\'
```

Compile on Windows Desktop

1. Download repository from https://github.com/stephen010x/capstone-rfsweep
2. Run install_cygwin.bat to install Cygwin.
3. From the CMD, run make
4. The binaries can found in bin\

Cross-Compile for Windows from Debian/Ubuntu

```
# Update apt
sudo apt update

# Install wine
sudo apt-get install wine

# Clone the project repository
git clone https://github.com/stephen010x/capstone-rfsweep.git --recurse-submodules
cd capstone-rfsweep

# Install Cygwin
wine install_cygwin.bat

# Build project
./winebuild.sh

# Once complete, the compiled output can be found in 'bin/'
```

Installing Script Dependancies (optional)

To use the scripts, you must install the script dependancies

Debian/Ubuntu Script Dependancies

```
# Update apt
sudo apt update

# Install python3 and pip
sudo apt-get install python3 pip --no-install-recommends

# Install python dependancies
sudo apt-get install python3-numpy python3-scipy python3-matplotlib --no-install-
          recommends
```

Windows CMD Script Dependancies

```
# Install python3
winget install Python.Python.3.14

# Install neccessary python libs
pip install numpy scipy matplotlib
```

Windows Desktop Script Dependancies

1. Install python from https://www.python.org/downloads/
2. Install neccessary python libs `pip install numpy scipy matplotlib`

Resources and Documentation

HackRF Documentation:

- https://pysdr.org/content/hackrf.html
- https://github.com/greatscottgadgets/hackrf/
- https://hackrf.readthedocs.io/en/latest/
- https://hackrf.readthedocs.io/en/latest/hackrf_one.html

Cygin Documentation:

- https://github.com/cygwin/cygwin/
- https://www.cygwin.com/

PCB Components Datesheets:

- https://www.pololu.com/file/0j450/a4988_dmos_microstepping_driver_with_translator.pdf
- https://pip.raspberrypi.com/documents/RP-008359-DS-raspberry-pi-zero-2-w-product-brief.pdf
- https://cdn-shop.adafruit.com/product-files/4115/4115_Zero4U_UserManual.pdf
- https://www.monolithicpower.com/en/documentview/productdocument/index/version/2/document_type/Datasheet/lang/en/sku/MP1584EN-LF-Z/document_id/204/?srsltid=AfmBOopY3mzal62NjcqQxcoeqAYdpTdsOmvjpPj1Y_Lt60M21O9yUpuI